RUTASEGURA GIRARDOT

Arquitectura, decisiones técnicas y registro de uso de inteligencia artificial

Maratón de Programación con Impacto Social (LCUP/CCPL)
Universidad de Cundinamarca · Ciclo de Profundización
4 de noviembre de 2026

Nicolás Morales Morales

Andrés Eduardo Jiménez Sabogal

Ivan David Miranda Castro

Presentación

Este documento reúne la descripción técnica de RutaSegura Girardot y el registro del apoyo recibido mediante herramientas de inteligencia artificial durante su desarrollo. La intención es dejar claro qué problema aborda la aplicación, cómo está construida, qué decisiones tomó el equipo y qué aspectos todavía requieren validación.

La IA se utilizó como apoyo para organizar tareas, proponer código, revisar errores y documentar el proceso. No sustituyó las decisiones del equipo: cada componente se revisó y se probó antes de incorporarlo al proyecto.

1. Problema que aborda la aplicación

La problemática asignada fue facilitar el desplazamiento seguro de las mujeres por el espacio público durante la noche, con una solución que pueda adaptarse a distintos municipios y que tenga un primer piloto en Girardot, Cundinamarca. El proyecto se relaciona con el ODS 5, sobre igualdad de género y libertad de movimiento, y con el ODS 11, sobre ciudades y comunidades inclusivas y seguras.

Se tuvo como referencia a una estudiante o trabajadora que camina de noche y utiliza un teléfono Android de gama media. Puede tener poca batería, datos limitados y una sola mano disponible; por eso se buscó que las acciones principales fueran sencillas y rápidas.

RutaSegura es una aplicación web que permite:

- Consultar zonas con poca iluminación o con reportes de la comunidad, además de ubicar lugares donde se puede pedir ayuda.
- Comparar una ruta rápida con una alternativa que prioriza condiciones de seguridad, mostrando los factores que influyen en la recomendación.
- Solicitar ayuda desde la aplicación, con acceso al número 123 y a contactos de emergencia.
- Consultar recorridos anteriores y calificarlos.
- Enviar reportes comunitarios que pueden contribuir al cálculo del puntaje después de pasar por moderación.
- Evitar que reportes falsos o malintencionados señalen injustamente a un sector.

2. Arquitectura y organización MVC

El proyecto sigue el patrón Modelo–Vista–Controlador (MVC), con una capa adicional de servicios para separar la lógica de negocio y la comunicación con APIs externas.

Capa	Responsabilidad	Ubicación / componentes
Modelo	Acceso a los datos. Es el único lugar donde se ejecutan consultas SQL, siempre parametrizadas.	src/models · PostgreSQL
Vista	Interfaz renderizada en el servidor. Las salidas se escapan para reducir el riesgo de XSS.	src/views · EJS, CSS, JavaScript y Leaflet
Controlador	Recibe las solicitudes, valida el flujo, llama a modelos y servicios y responde con una	src/controllers · auth, map, route, trip, alert, contact, incident y admin

	vista o JSON.	
Servicios	Contienen reglas del negocio e integración con servicios externos. El cálculo del puntaje se mantiene en funciones probables y testeables.	src/services · routePlanner, safetyScore, routing, overpass, geocode, crypto y notification
Middleware	Aplica autenticación, permisos por rol, validaciones, límites de solicitudes y manejo centralizado de errores.	auth, requireRole, validators, rateLimits y errorHandler

El recorrido de una solicitud comienza en el navegador de la usuaria. La comunicación se realiza mediante HTTPS y una cookie con JWT protegida como httpOnly. Las rutas web y de API pasan por los middleware correspondientes; después llegan al controlador, que utiliza los servicios y modelos necesarios.

El controlador no contiene consultas SQL. Los modelos usan parámetros como $1 y $2 para evitar concatenar directamente datos recibidos desde el cliente. En las vistas EJS se utiliza <%= %> para escapar el contenido mostrado.

3. Tecnologías seleccionadas

- Node.js y Express 4: se eligieron por ser tecnologías ligeras y conocidas por el equipo.
- EJS: permite renderizar las páginas desde el servidor, sin depender de un framework adicional de frontend.
- PostgreSQL en Neon: cumple el requisito de base de datos del ciclo y ofrece un nivel gratuito que permitió trabajar sin instalar un servidor local de PostgreSQL.
- Leaflet y OpenStreetMap: se utilizan para mostrar el mapa y sus capas.
- OpenRouteService y OSRM FOSSGIS: permiten calcular recorridos peatonales y contar con una alternativa de respaldo.
- Render: se utilizó para el despliegue final después de intentar primero con Azure.

4. Modelo de datos

La base de datos está definida en db/schema.sql y contiene 13 tablas. Su estructura contempla los siguientes elementos:

- cities: permite configurar más de una ciudad.
- users: almacena las cuentas, el rol user/admin y el consentimiento relacionado con la Ley 1581.
- emergency_contacts: guarda los teléfonos y correos de los contactos de emergencia cifrados.
- risk_zones y safe_points: registran zonas de riesgo y puntos de apoyo, indicando si el origen es demo, OSM o comunidad.
- trips: almacena los recorridos y su geometría en formato GeoJSON dentro de un campo JSONB.
- trip_ratings y segment_scores: guardan las calificaciones y el puntaje agregado por celdas de aproximadamente 100 metros.
- incidents: registra reportes con estados pending, approved y rejected.
- comments y share_links: quedan preparadas para funciones futuras.
- alerts y security_events: permiten registrar alertas y eventos relevantes de seguridad.

Las coordenadas se manejan como latitud y longitud, sin PostGIS. Las distancias se calculan en el servidor mediante la fórmula de Haversine. Para el alcance actual, esta solución resulta suficiente para el piloto en una ciudad.

5. APIs externas y datos abiertos

Servicio	Uso	Motivo	Comportamiento si falla
OpenRouteService (foot-walking)	Calcula rutas a pie y hasta tres alternativas.	Ofrece un perfil peatonal.	Se intenta con OSRM.
OSRM FOSSGIS (routed-foot)	Respaldo de ruteo; actualmente es el servicio activo.	Es gratuito y no requiere clave. El OSRM público de project-osrm.org solo calcula rutas en carro.	Se muestra un mensaje claro si no responde.
Overpass / OpenStreetMap	Consulta estaciones de policía, comercios 24 horas y luminarias.	Permite aprovechar datos abiertos; se obtuvieron 28 puntos en Girardot.	Se consulta un servidor espejo y, si es necesario, se usan datos de demostración.
Nominatim	Busca lugares por nombre.	Complementa la selección directa sobre el mapa.	Se muestra una lista vacía con un mensaje.
Teselas OSM y Leaflet	Muestra el mapa base.	Son herramientas abiertas y gratuitas.	No se definió un respaldo adicional.

Para reducir el riesgo de SSRF, el servidor solo realiza solicitudes a hosts externos definidos previamente. Las coordenadas se validan como números y no se aceptan URLs enviadas por el cliente. Los resultados de Overpass se guardan en caché en safe_points mediante npm run osm:sync, en lugar de consultar el servicio en cada petición.

6. Cálculo del puntaje de seguridad

El servicio src/services/safetyScoreService.js toma puntos de la ruta cada 30 metros y calcula un puntaje entre 0 y 100. Los factores definidos para esta versión son:

Factor	Efecto sobre el puntaje
Tramo de 30 m dentro de una zona de riesgo	−0,6 × nivel de riesgo (1 a 5), con una penalización máxima de −60.
Porcentaje del recorrido sin punto seguro ni luminaria a menos de 150 m	Puede restar hasta 20 puntos.
CAI, estación de policía o comercio abierto 24 horas a menos de 100 m	Suma 5 puntos por cada uno, hasta un máximo de 15.
Promedio de estrellas de las celdas recorridas	Puede aportar entre −10 y +10 puntos.
Tiempo superior a 1,3 veces el de la ruta rápida	Resta 1 punto por cada 1 % adicional.

Para escoger una alternativa, primero se solicitan rutas al servicio de ruteo. Si la mejor opción todavía atraviesa una zona de riesgo, el sistema intenta generar dos desvíos con puntos intermedios a ambos lados de la zona más comprometida. Después selecciona la ruta con mayor puntaje, siempre que no supere en más de 40 % el tiempo de la ruta rápida.

La explicación que recibe la usuaria se refiere a condiciones concretas, por ejemplo, poca iluminación o cercanía a una estación de policía. No se mencionan barrios ni se etiqueta a una comunidad completa.

Durante el desarrollo se modificó una decisión inicial del CLAUDE.md. La propuesta original normalizaba la penalización por el número de puntos de la ruta; el equipo decidió calcularla por metro expuesto, porque el promedio podía reducir el efecto de una zona peligrosa en recorridos largos.

Es importante aclarar que los pesos son una primera propuesta y todavía no cuentan con validación estadística. Además, parte de la información de riesgo es de demostración. Por eso, el puntaje sirve como referencia para tomar una decisión, no como predicción del delito ni como garantía de seguridad.

7. Seguridad y protección de datos

Se tomaron como referencia riesgos del OWASP Top 10 y las obligaciones relacionadas con la Ley 1581 de 2012. Las medidas implementadas incluyen:

- Control de acceso: requireAuth y requireRole('admin') protegen las rutas. Las consultas de recorridos y contactos se filtran por user_id; se probó un intento de acceso a información de otra cuenta y la respuesta fue 404.
- Contraseñas y sesiones: las contraseñas se procesan con bcrypt (costo 11). La sesión utiliza JWT HS256 con duración de dos horas, en una cookie httpOnly y sameSite=strict; en producción también se activa secure.
- Cifrado de contactos: los teléfonos y correos se cifran con AES-256-GCM. La clave DATA_KEY se configura mediante variables de entorno.
- Conexiones: se utiliza HTTPS con HSTS y la conexión a Neon valida el certificado mediante sslmode=verify-full.
- Validación de entradas: express-validator revisa coordenadas, tipos permitidos y longitudes máximas. Las consultas SQL son parametrizadas.
- Protección de vistas: EJS escapa las salidas y se probó que una etiqueta script en un reporte se muestre como texto.
- Configuración: Helmet aplica una CSP estricta, CORS se limita a APP_URL y no se muestran trazas internas a la usuaria. El archivo .env se mantiene fuera del repositorio y del paquete de despliegue.
- Límite de intentos: se permiten hasta 10 intentos fallidos cada 15 minutos en inicio de sesión y registro. La contraseña debe tener al menos 10 caracteres con letras y números.
- Auditoría: security_events registra inicios de sesión fallidos, accesos denegados, alertas, moderación, límites de intentos y eliminación de cuentas.
- CSRF: se utiliza sameSite=strict y la API acepta únicamente solicitudes JSON.

La aplicación solicita consentimiento para el uso de la ubicación, aplica minimización de datos y permite eliminar la cuenta junto con sus datos desde la sección “Mi red”. Los datos personales reales y las claves de conexión no se compartieron con la herramienta de IA.

8. Experiencia de uso y accesibilidad

La línea visual del proyecto se definió como “farol”: una interfaz nocturna, cálida y de acompañamiento. Se utiliza un fondo oscuro inspirado en la noche del Magdalena (#0E1A1F), un tono ámbar (#F2B33D) y rojo para las acciones de emergencia (#B3261E). Los títulos usan Fraunces y el texto Atkinson Hyperlegible, una tipografía pensada para facilitar la lectura.

El puntaje se presenta tanto con diez faroles encendidos como con un valor numérico. De esta manera, la información no depende únicamente del color o de un elemento visual.

Entre las medidas de accesibilidad previstas bajo WCAG 2.1 AA están:

- Contraste mínimo de 4,5:1 en los tokens de color.
- Etiquetas para los campos y mensajes de error asociados mediante aria-describedby.
- Mensajes de estado del mapa anunciados con aria-live.
- Foco visible de 3 px y enlace para saltar directamente al contenido.
- Idioma es-CO, objetivos táctiles de al menos 44 px y texto de 17 px.
- Modo noche automático o manual y respeto por prefers-reduced-motion.
- Diferenciación de rutas por color y estilo de línea, además de una lista textual de zonas y puntos como alternativa al mapa.

El onboarding se divide en tres pasos: explicación de las rutas, instrucciones para pedir ayuda y consentimiento. La acción de ayuda se puede activar con un toque y tiene tres segundos para cancelarse. Trazar una ruta requiere seleccionar origen y destino y pulsar “Trazar”.

Durante un recorrido se puede configurar un tiempo de inactividad entre 1 y 60 minutos. Si no hay interacción, aparece la pregunta “¿Estás bien?” junto con una alarma; si no hay respuesta en 20 segundos, se genera una alerta con el motivo inactividad.

Prueba de usabilidad pendiente

Esta tabla debe completarse con la prueba de una persona de otro equipo. No se registraron resultados en los documentos de trabajo recibidos.

Tarea	Tiempo	Problemas observados	Corrección aplicada
Trazar una ruta segura	4 Segundos	En el navegador de escritorio el mapa no cargaba: OpenStreetMap bloqueaba las teselas ("Access blocked", error 403). En el celular sí se veía.	Se ajustó la cabecera Referrer-Policy a strict-origin-when-cross-origin. El mapa carga en escritorio y celular.
Pedir ayuda	2 Segundos	La alarma no sonaba sola y, cuando vencía el "¿Estás bien?", no había forma de avisar a los contactos.	Alarma automática al confirmar la alerta y botones "Avisar por WhatsApp" con el mensaje y la ubicación ya escritos, también en la alerta por inactividad.
Calificar un recorrido	4 Segundos	Ninguno	No requirió

9. Escalabilidad, despliegue y sostenibilidad

La aplicación está preparada para incorporar otros municipios. Para agregar uno se registra su centro y recuadro geográfico en la tabla cities y se ejecuta npm run osm:sync. La lógica no está limitada de forma fija a Girardot.

Como el servidor no mantiene estado de sesión en memoria y utiliza JWT, es posible agregar instancias sin modificar la lógica principal. Si el proyecto crece, se contempla incorporar PostGIS con índices espaciales, una caché para rutas frecuentes y una cola para el envío de notificaciones.

El despliegue disponible está en https://rutasegurasgirardot.onrender.com, mediante un servicio gratuito de Render conectado a GitHub. Inicialmente se intentó utilizar Azure, tal como se esperaba por el reglamento. Sin embargo, Azure for Students rechazó las regiones East US, East US 2 y otras por la política “Allowed resource deployment regions”. En una región permitida, la creación del servicio tampoco fue posible debido a que la cuota de máquinas F1 y B1 estaba en cero. Solicitar una ampliación a soporte excedía el tiempo disponible de la maratón, que era de 2,5 horas. Por esa razón se utilizó Render como alternativa y se dejó constancia de ello.

El costo actual reportado es de $0, con Render Free, Neon Free y APIs abiertas. Hay límites que deben tenerse en cuenta: Render suspende el servicio después de 15 minutos sin actividad y el primer arranque puede tardar alrededor de 50 segundos; OpenRouteService permite hasta 2.000 rutas diarias en su nivel gratuito.

A nivel social, el siguiente paso sería coordinar con la Alcaldía, la Secretaría de la Mujer y la Policía para sustituir los datos de demostración por información oficial y validada. También se propone trabajar con universidades para apoyar la moderación de reportes.

10. Limitaciones conocidas y trabajo futuro

Las limitaciones identificadas durante las pruebas son las siguientes:

- Las zonas de riesgo contienen datos de demostración y deben validarse con la comunidad y las autoridades antes de utilizar la aplicación en un contexto real.
- Una aplicación web no puede aumentar el volumen físico del teléfono. La alarma depende del volumen configurado por la usuaria, situación que se explica durante el onboarding.
- Cuando el navegador pasa a segundo plano, puede pausar el seguimiento y el temporizador; el Wake Lock tampoco está garantizado.
- OpenStreetMap tiene una cobertura reducida de luminarias en Girardot: se encontraron 16 registradas.
- El aviso por Telegram se simula si no se cuenta con credenciales. Los enlaces de WhatsApp y SMS se abren desde el teléfono de la usuaria.
- El rol está incluido en el JWT, por lo que un cambio de permisos puede tardar hasta dos horas en reflejarse, cuando vence el token.

Como trabajo futuro se plantean la ubicación compartida en tiempo real mediante enlaces con vencimiento, un heartbeat del servidor durante el recorrido, notificaciones reales por Telegram o SMS, rutas construidas con aportes comunitarios, incorporación de datos oficiales de Policía y Alcaldía, análisis del riesgo por hora con modelos de aprendizaje automático evaluados para detectar sesgos y, más adelante, una aplicación móvil nativa.

11. Uso de inteligencia artificial durante el desarrollo

La herramienta utilizada fue Claude Code, con el modelo Claude Opus 5.5 desde la extensión de Visual Studio Code. El equipo registró los prompts en registro-ia/prompts.md y los archivos intervenidos en registro-ia/archivos.md. El registro que sigue resume los momentos principales del proceso.

11.1. Cómo se utilizó y qué decisiones tomó el equipo

Fase 0. Estructura inicial

Aporte de la herramienta: Se le pidió a la herramienta que leyera el archivo CLAUDE.md y propusiera un plan de trabajo. A partir de esa revisión generó la estructura MVC, package.json,

app.js con Helmet y una política CSP, CORS restringido, cookie-parser, manejo centralizado de errores, .env.example y .gitignore.

Revisión y decisión del equipo: El equipo eligió Azure en lugar de Render porque el reglamento del ciclo priorizaba AWS, GCP o Azure. También se detuvo una instrucción que habría cerrado todos los procesos de Node del computador; se reemplazó por detener únicamente el servidor de pruebas. Se aceptó bcryptjs en lugar de bcrypt para evitar compilaciones nativas en Windows con Node 24. Se estimó un ahorro de unos 30 minutos en tareas de configuración.

Fase 1. Base de datos y autenticación

Aporte de la herramienta: Ante la pregunta sobre PostgreSQL y Neon, la IA explicó que Neon ofrece PostgreSQL administrado en la nube y que no era necesario instalar un servidor local. Propuso el esquema de 13 tablas, datos de demostración, inicialización de usuarios de prueba, modelos User y City, autenticación, validaciones, límites de intentos y vistas de ingreso y registro.

Revisión y decisión del equipo: El equipo decidió no compartir la URL de la base de datos en el chat. También descartó instalar la CLI de Neon, ya que bastaba con configurar la URL en .env y utilizar el driver pg. Se cambió sslmode=require por verify-full para validar el certificado. Las pruebas incluyeron el rechazo de contraseñas débiles, el registro con cookie HttpOnly, la redirección al onboarding, la protección de /map sin sesión y mensajes genéricos ante un inicio de sesión fallido. Se documentó que un cambio de rol puede tardar hasta dos horas en aplicarse porque el rol forma parte del JWT.

Fase 2. Mapa y datos abiertos

Aporte de la herramienta: La herramienta implementó GET /api/map/layers, los modelos RiskZone y SafePoint, la integración con Overpass y el comando npm run osm:sync. También preparó el mapa con Leaflet servido desde node_modules y una lista textual de capas para quienes no puedan utilizar el mapa.

Revisión y decisión del equipo: La primera consulta a Overpass agotó el tiempo de espera. El equipo agregó un servidor espejo y logró recuperar datos reales: 5 estaciones de policía, 16 luminarias y 7 lugares abiertos las 24 horas en Girardot. La cobertura de luminarias sigue siendo limitada, por lo que se combinan esos datos con reportes comunitarios moderados.

Fase 3. Ruteo y puntaje

Aporte de la herramienta: A partir de la fórmula descrita en el CLAUDE.md, la IA generó geoUtils, safetyScoreService, routingService, routePlanner, búsqueda de lugares con Nominatim y 11 pruebas unitarias con node:test.

Revisión y decisión del equipo: Durante la revisión se detectó que el OSRM público de project-osrm.org solo calcula rutas en carro, incluso si se solicita el perfil foot. Se cambió al servidor OSRM de FOSSGIS con perfil peatonal. También se modificó la penalización del puntaje para calcularla por metro expuesto y no normalizarla por cantidad de puntos. Como las alternativas de la API tendían a pasar por zonas similares, se añadieron desvíos con puntos intermedios alrededor de la zona de mayor riesgo. El equipo corrigió además algunos detalles de redacción de la explicación mostrada a la usuaria.

Fase 4. Recorridos, historial, calificaciones y emergencia

Aporte de la herramienta: Se generaron los modelos Trip, TripRating, SegmentScore, Alert y EmergencyContact. También se implementó el cifrado AES-256-GCM de teléfonos y correos, el botón de ayuda con tres segundos para cancelar, enlaces de WhatsApp con mensaje preparado, alarma mediante Web Audio API y Screen Wake Lock.

Revisión y decisión del equipo: Como no había credenciales de Telegram, el equipo decidió mostrar que el envío era simulado en lugar de indicar que la alerta había sido enviada. Se probó el flujo de trazar, iniciar, generar una alerta, finalizar, calificar e ingresar al historial. En una prueba, el puntaje de una misma ruta pasó de 74 a 84 después de la retroalimentación. También se verificó que solicitar el recorrido de otra usuaria devolviera 404 y que el teléfono se almacenara cifrado. Se documentó que una web no puede subir el volumen del celular ni garantizar su funcionamiento en segundo plano.

Fase 6. Control de inactividad

Aporte de la herramienta: La herramienta ayudó a completar esta función al detectar que la pantalla “Mi red” permitía configurar la inactividad, aunque todavía no existía el comportamiento correspondiente. Se añadió un temporizador que se reinicia con cada toque, un diálogo de confirmación de 20 segundos, una alarma y la generación de alertas con el motivo inactividad.

Revisión y decisión del equipo: El audio debe activarse con el primer toque porque así lo exige el navegador. También se dejó documentado que el temporizador puede pausarse si el navegador queda en segundo plano.

Fase 7. Reportes y moderación

Aporte de la herramienta: Como el reglamento exigía al menos dos roles con permisos diferentes, se adelantó el desarrollo de reportes anónimos desde el mapa mediante POST /api/incidents y un panel /admin/incidents para que la administradora apruebe o rechace cada reporte.

Revisión y decisión del equipo: Las pruebas confirmaron que una usuaria recibe 403 al intentar entrar al panel de moderación, que la aprobación crea una zona comunitaria y que una etiqueta script en la descripción se muestra escapada. La revisión humana es importante para evitar que reportes malintencionados o difamatorios señalen un sector.

Fase 8. Despliegue

Aporte de la herramienta: La IA acompañó la creación del servicio en Azure e interpretó los errores que aparecieron durante la configuración. Identificó restricciones de regiones y cuotas, propuso Render como alternativa, ayudó a preparar el commit y el push y verificó que .env no se incluyera en el repositorio.

Revisión y decisión del equipo: El equipo revisó la política de la suscripción sin modificarla, creó un repositorio privado en GitHub e intentó primero completar el despliegue en Azure. Al no ser posible resolver las cuotas dentro del tiempo disponible, se aceptó Render como alternativa y se documentó el motivo. En producción se probaron el login, las cookies Secure y HttpOnly, HSTS, CSP, las capas del mapa, el cálculo de rutas y la carga de las pantallas. La primera sugerencia de región de Azure tuvo que corregirse con la política real de la cuenta.

11.2. Resumen de herramientas y aportes

Herramienta / uso	Solicitud o contexto	Resultado	Revisión del equipo
Claude Code (Opus 5.5): planificación, MVC, código, pruebas y documentación	“leee claude.md” y “dale”	Estructura de la aplicación y 11 pruebas unitarias.	Se ajustó la fórmula del puntaje, se cambió el perfil OSRM, se bloqueó un comando destructivo y se cambió el proveedor de despliegue.

Claude Code: explicación de Neon y PostgreSQL	"¿Qué es ese neon.tech?"	Orientación para utilizar PostgreSQL administrado en la nube.	Se descartó instalar la CLI y no se compartió la URL de la base de datos.
Claude Code: diagnóstico de Azure	Capturas de los errores de configuración	Identificación de restricciones de región y cuota.	No se modificó la política de la cuenta; se utilizó Render como alternativa.

Si el equipo utilizó otras herramientas de IA, debe agregarlas aquí antes de entregar el documento: [herramienta, finalidad, solicitud realizada, resultado y revisión del equipo].

12. Criterios éticos aplicados al uso de IA y al proyecto

El equipo tomó como referencia el Marco Ético para la Inteligencia Artificial en Colombia (2021). Estos principios se reflejan tanto en el desarrollo como en el funcionamiento previsto de RutaSegura:

Principio	Aplicación en el proyecto
Transparencia y explicación	Los usos de IA se registraron en este documento y en registro-ia/. La aplicación explica por qué recomienda una ruta y qué factores intervienen en el puntaje.
Privacidad	Se solicita consentimiento para la ubicación, se cifran los contactos de emergencia y se permite eliminar la cuenta. No se compartieron datos reales ni secretos con la IA.
Control humano de las decisiones	La herramienta propuso alternativas, pero el equipo decidió el proveedor de nube, la base de datos y los cambios en la fórmula. La usuaria elige la ruta y una persona modera los reportes.
Seguridad	Se incorporaron y probaron controles como límites de intentos, CSP, protección frente a IDOR y XSS, cifrado y HTTPS.
Responsabilidad	El equipo revisó las propuestas, probó las funciones y asume la responsabilidad por el código entregado.
No discriminación	Las explicaciones hablan de condiciones como iluminación y reportes, no de barrios. Los datos de riesgo de demostración se identifican como tales.
Inclusión	Se contemplan criterios WCAG 2.1 AA, tipografía legible, uso con una mano, modo noche y alternativas textuales al mapa.
Derechos de niñas, niños y adolescentes	La versión actual está dirigida a mujeres adultas y no recoge datos de menores. Una versión para adolescentes requeriría revisar las condiciones de consentimiento de sus acudientes.

Beneficio social	La propuesta busca aportar a los ODS 5 y 11, facilitando información y acompañamiento para desplazamientos nocturnos. El servicio se mantiene sin costo en la configuración actual y puede adaptarse a otros municipios.

13. Balance del uso de inteligencia artificial

En qué fue más útil

El mayor aporte estuvo en acelerar la estructura MVC, las tareas repetitivas de seguridad —como validaciones, cabeceras y cifrado— y la documentación. También permitió identificar problemas que podían pasar inadvertidos durante el tiempo limitado de la maratón, como la diferencia entre el ruteo en carro y el peatonal en el servicio OSRM, o una opción de configuración que todavía no tenía una función implementada.

En qué fue necesario tener más cuidado

La herramienta no conocía las restricciones reales de la suscripción de Azure y al principio asumió que algunas regiones estaban disponibles. La configuración de servicios externos, las cuotas y los permisos siempre tuvieron que verificarse directamente por el equipo.

Riesgos y medidas tomadas

- Estigmatización de sectores: se explican factores observables, se exige moderación humana y se identifican los datos de demostración.
- Privacidad de la ubicación: se solicita consentimiento, no se conserva un rastreo continuo como historial automático y se ofrece la eliminación de la cuenta.
- Falsa sensación de seguridad: en las pantallas clave se aclara que la aplicación es una herramienta de apoyo, no una garantía, y que en una emergencia se debe llamar al 123.

Cierre

RutaSegura Girardot es un prototipo funcional que integra mapas, ruteo peatonal, información comunitaria, mecanismos de emergencia y controles de seguridad. Durante la maratón se priorizó que las funciones principales pudieran probarse y que las limitaciones quedaran registradas, en lugar de presentar como terminadas capacidades que todavía dependen de servicios externos o de validación en campo.

La inteligencia artificial ayudó a avanzar más rápido, pero no reemplazó el criterio técnico del equipo. Las decisiones sobre arquitectura, datos, seguridad y despliegue se revisaron durante el proceso. Antes de utilizar la aplicación fuera de una demostración, será necesario validar los datos con la comunidad y las autoridades, completar las pruebas de usabilidad y resolver las limitaciones de funcionamiento en segundo plano y de notificaciones.